

grille_1d.py : Guide

1 Objectif et contexte

1.1 Le problème résolu

Segmenter un texte manuscrit annoté (`*_with_BM.json`) en lignes puis en mots, en trouvant automatiquement les deux hyperparamètres de la méthode à deux passes (`eps_y` pour les lignes, `eps_x` pour les mots), et en mesurant la qualité du résultat par l'ARI (Adjusted Rand Index) contre les labels BM réels.

1.2 L'intérêt dans le projet

C'est le tout premier script écrit pour apprendre à manipuler DBSCAN sur ce type de données, avant `grille_2d.py`. L'objectif initial n'était pas de comparer méthodiquement une recherche séparée des paramètres à une recherche conjointe, mais de commencer simplement : chercher `eps_y` seul, puis `eps_x` seul, pour comprendre le comportement de l'algorithme avant de complexifier vers une recherche jointe des deux paramètres.

1.3 Concepts clés à connaître avant de lire le code

- **Méthode à deux passes (pipeline $Y \rightarrow X$)** : DBSCAN sur Y seul pour détecter les lignes, puis DBSCAN sur X à l'intérieur de chaque ligne pour détecter les mots.
- **ARI (Adjusted Rand Index)** : mesure d'accord entre deux partitions (ici, mots prédits et BM réels), valant 1 pour un accord parfait et proche de 0 pour un résultat comparable au hasard.
- **Grille 1D** : on ne fait varier qu'un seul paramètre à la fois. Comme on a `eps_y` et `eps_x`, on les cherche l'un après l'autre : d'abord `eps_y` seul (sans regarder l'ARI, juste le nombre de lignes), puis `eps_x` seul (à `eps_y` fixé, en maximisant l'ARI). C'est ce qui distingue ce script de `grille_2d.py`, qui teste le couple (`eps_y`, `eps_x`) simultanément.

2 Structure du code

2.1 Architecture générale

Le script est linéaire (pas de classes), organisé en étapes numérotées.

Étape	Rôle
0	Lecture des arguments (<code>fichier</code> , <code>minS_y</code> , <code>minS_x</code>)
1	Chargement du JSON, filtrage de la ponctuation (<code>BM == "P"</code>)
2	<code>copie()</code> , une copie profonde pour ne pas polluer les essais suivants
3	<code>deux_passes()</code> , la fonction cœur, DBSCAN Y puis X, calcule l'ARI
5	Grille sur <code>eps_y</code> seul, détection de plateaux stables
6	Grille sur <code>eps_x</code> seul (à <code>eps_y</code> fixé), maximisation de l'ARI
7	Impact de <code>min_samples</code> sur l'ARI (comparatif)
8 et 9	Sauvegardes CSV et figures d'analyse
final	Figures texte (BM réel, MotID, LigneID) et résumé terminal

3 Explication détaillée par étape

3.1 La fonction clé (étape 3)

Prend (`eps_y`, `eps_x`, `mSy`, `mSx`), applique DBSCAN sur *Y* puis, pour chaque ligne, DBSCAN sur *X*, et calcule l'ARI en comparant le résultat aux **BM** réels.

```

# Passe 1 : separation des lignes
labels_ligne = DBSCAN(eps_y, mSy).fit_predict(coords_Y)
lignes_uniques = valeurs distinctes de labels_ligne, sauf le bruit (-1)

# Initialisation avant la passe 2
labels_mot = tableau rempli de -1 (bruit par default)
mot_global = 0 # compteur qui garantit des MotID uniques entre lignes

pour chaque ligne_id dans lignes_uniques :
    idx = indices des points appartenant a cette ligne
    coords_X_ligne = coordonnee X de ces points seulement

    # Passe 2 : separation des mots, a l'interieur de la ligne
    labels_local = DBSCAN(eps_x, mSx).fit_predict(coords_X_ligne)
    mots_locaux = valeurs distinctes de labels_local, sauf le bruit

    # Report des labels locaux vers le tableau global, avec decalage
    pour chaque point i de cette ligne :
        si labels_local[i] != -1 :
            labels_mot[idx[i]] = mot_global + labels_local[i]

    # Le decalage mot_global evite que le mot 0 de deux lignes
    # differentes ne partagent le meme identifiant
    si mots_locaux n'est pas vide :
        mot_global += plus grand numero de mots_locaux + 1

# Report des labels sur les points d'origine, via l'identifiant N
pour chaque point du document :
    LigneID du point = labels_ligne correspondant
    MotID du point = labels_mot correspondant

# Agregation au niveau du stroke, par vote majoritaire
# (un stroke contient plusieurs points, qui peuvent porter des
# labels legerement differents a la frontiere de deux clusters)
pour chaque stroke :
    LigneID du stroke = label le plus frequent parmi ses points
    MotID du stroke = label le plus frequent parmi ses points

# Calcul de l'ARI, en excluant les strokes classes bruit
strokes_valides = strokes dont le MotID n'est pas -1
ari = ARI(BM reels, MotID) sur les strokes_valides

n_mots = nombre de MotID distincts (sans compter le bruit)
n_lignes = nombre de lignes_uniques
retourner (n_lignes, n_mots, ari)

```

Cette fonction sert à la fois à l'étape 6 (recherche de `eps_x`) et à la génération finale des figures.

3.2 Grille `eps_y` et notion de plateau stable (étape 5)

On teste `eps_y` de 0.05 à 1.0 par pas de 0.01, soit 89 valeurs, en appliquant DBSCAN sur *Y* seul (pas la méthode complète, puisqu'on n'a pas besoin de l'ARI ici, juste du nombre de lignes).

On obtient une courbe reliant **eps_y** au nombre de lignes détecté :

eps_y	n_lignes
0.05	42
0.06	18
...	...
0.80	7 (le plateau commence)
0.82	7
...	...
1.00	7 (le plateau finit)

Un plateau est une plage de **eps_y** consécutifs qui donnent le même nombre de lignes pendant au moins **MIN_PLATEAU** (soit 3) valeurs d'affilée. L'idée est qu'un vrai nombre de lignes correspond à une plage stable, pas à une valeur isolée qui pourrait être un artefact de bruit numérique.

Comment on choisit le bon plateau. Parmi tous les plateaux détectés, on ne garde que ceux dont le nombre de lignes tombe dans une plage jugée cohérente avec le nombre de mots réels, entre $n_bm_reels // 12$ et $n_bm_reels // 4$. Puis, parmi ces plateaux cohérents, on choisit celui dont le nombre de lignes est le plus proche de $\sqrt{n_bm_reels}$.

D'où viennent les bornes 12 et 4. Ce ne sont pas des valeurs arbitraires : elles traduisent une hypothèse sur le nombre de mots par ligne. En notant L le nombre de lignes et N le nombre de mots réels, la condition

$$\frac{N}{12} \leq L \leq \frac{N}{4}$$

s'inverse, en divisant par L , en

$$4 \leq \frac{N}{L} \leq 12.$$

Autrement dit, on suppose qu'une ligne d'écriture manuscrite contient typiquement entre 4 et 12 mots. En dessous de 4 mots par ligne, le plateau est jugé suspect : il correspondrait plutôt à une sur-segmentation, un texte découpé en trop de lignes. Au-dessus de 12 mots par ligne, le plateau est également jugé suspect : il correspondrait à une sous-segmentation, plusieurs lignes réelles fusionnées en une seule.

Sur l'exemple du fichier de référence, avec $N = 55$ mots réels, la plage devient $55//12 = 4$ à $55//4 = 13$ lignes. Le plateau à 7 lignes observé tombe bien dans cette fourchette.

D'où vient ce $\sqrt{n_bm_reels}$. C'est un second raisonnement, appliqué une fois la plage grossière filtrée, pour départager les plateaux restants. Si on a N mots répartis sur L lignes, et qu'en moyenne chaque ligne contient à peu près le même nombre de mots (N/L), chercher un équilibre $L \approx N/L$ donne $L^2 \approx N$, donc $L \approx \sqrt{N}$. Sur l'exemple du fichier de référence, $\sqrt{55} \approx 7.41$, proche du plateau à 7 lignes observé, mais ce n'est qu'une heuristique approximative : l'hypothèse d'un nombre de mots égal par ligne n'est pas toujours vraie.

eps_y_opt est fixé à la médiane du plateau retenu, ni au début ni à la fin, pour rester au centre de la zone stable.

3.3 Grille **eps_x** et zones de regroupement (étape 6)

À **eps_y_opt** fixé, on teste **eps_x** de 0.10 à 3.0 par pas de 0.05, cette fois via la fonction complète (donc avec calcul de l'ARI). On retient **eps_x_opt** comme la valeur qui maximise l'ARI.

En parallèle, on classe chaque valeur de `eps_x` testée en trois zones, selon le nombre de mots obtenu comparé au nombre de mots réels (`n_bm_reels`) :

- sur-regroupement : nombre de mots inférieur à 80 % du réel, ce qui signifie que `eps_x` est trop grand et que plusieurs mots ont fusionné en un seul.
- zone optimale : nombre de mots compris entre 80 % et 120 % du réel.
- sous-regroupement : nombre de mots supérieur à 120 % du réel, ce qui signifie que `eps_x` est trop petit et qu'un mot a été coupé en plusieurs morceaux.

Ces zones servent surtout de diagnostic visuel (la figure du milieu, avec les zones ombrées) et de statistique de synthèse (`pct_zone_optimale`, entre autres), pas directement au choix de `eps_x_opt`, qui suit uniquement l'ARI maximal.

3.4 Impact de `min_samples` (étape 7)

C'est un comparatif indépendant : pour `minS` $\in \{2, 3, 5\}$, on refait toute la grille `eps_x` (à `eps_y_opt` fixé) et on trace l'ARI obtenu. Cela sert uniquement à visualiser la sensibilité du résultat à ce paramètre, et ne modifie ni `eps_x_opt` ni `ari_opt`, qui restent ceux calculés à l'étape 6 avec les `minS_y/minS_x` donnés en argument.

3.5 Sorties (étapes 8 et 9)

Trois CSV (`grille_epsy`, `grille_epsx`, `impact_minS`), un CSV de synthèse à une ligne par scripteur, une figure à trois panneaux (`eps_y` contre `n_lignes`, `eps_x` contre ARI et `n_mots`, impact de `min_samples`), trois figures texte (`BM_reel`, `MotID`, `LigneID`), et un résumé terminal sauvegardé en `.txt`.